Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)

BAB I
PENDAHULUAN

1.1 Penjelasan Dataset

Dataset ini berjudul "SGEMM GPU kernel performance" atau Single precision GEneral Matrix Multiply) pada GPU yaitu waktu untuk melakukan perkalian matriks dengan ukuran 2048 x 2048 menggunakan beberapa kombinasi parameter kernel yang berbeda. Dataset ini memiliki total 241.600 baris, terdapat 14 fitur input ada: (*Konfigurasi Tiling & Workgroup)* MWG, NWG, KWG, MDIMC, NDIMC, MDIMA, NDIMB; (*Optimasi Kernel)* KWI (unrolling), VWM, VWN; (*Akses Memori)* STRM, STRN (stride), SA, SB (caching). Target Output (4 Kolom), yaitu: Run1, Run2, Run3, dan Run4 (waktu eksekusi dalam milidetik). Dataset ini cocok untuk tugas regresi karena variabel targetnya berupa angka kontinu (waktu eksekusi dalam milidetik), bukan kategori. Nilainya juga memiliki rentang yang sangat luas, dari belasan hingga ribuan milidetik, sehingga membutuhkan model yang mampu memprediksi nilai secara akurat. Selain itu, hubungan antara parameter konfigurasi kernel dan waktu eksekusi bersifat kompleks dan bisa non-linear, sehingga regresi tepat digunakan untuk memodelkan hubungan tersebut secara matematis.

Performa kernel GPU tidak hanya dipengaruhi oleh satu parameter tunggal, melainkan oleh interaksi antar parameter seperti ukuran tiling, konfigurasi workgroup, serta pola akses memori. Interaksi ini dapat menghasilkan interaction effect antar fitur dan potensi perilaku non-linear, sehingga hubungan antara variabel input dan waktu eksekusi tidak selalu berbentuk linier sederhana. Oleh karena itu, pendekatan regresi digunakan untuk memodelkan hubungan numerik antar variabel secara kuantitatif dan sistematis. Penggabungan empat pengukuran runtime (Run1-Run4) melalui rata-rata bertujuan untuk mengurangi variasi eksperimen (experimental noise) dan meningkatkan stabilitas estimasi target. Dalam eksperimen performa komputasi, averaging antar run merupakan praktik umum untuk memperoleh estimasi waktu eksekusi yang lebih representatif dan reliabel.

1.2 Preprocessing & Pipeline Preprocessing

Preprocessing data dalam machine learning melibatkan proses mengubah data mentah yang tidak terstruktur menjadi format yang terorganisasi dengan baik dan dapat digunakan untuk analisis serta pemodelan (GeeksforGeeks, 2026). Proses ini bertujuan untuk mengurangi noise, mengatasi inkonsistensi, dan meningkatkan kualitas data agar model machine learning dapat bekerja lebih efektif dan efisien (DQLab, 2025). Preprocessing dalam kode ini dilakukan lewat 2 tahap yaitu:

1. pada saat transformasi menggunakan :

Tabel 1. 1 Transformasi

```
y = df.iloc[:, 14:].mean(axis=1)
```

dimana pada tahap ini, empat kolom runtime (Run1-Run4) digabungkan dengan cara menghitung rata-rata setiap baris sehingga menghasilkan satu variabel target kontinu. Proses ini dilakukan setelah data dibaca dan sebelum model dilatih. Transformasi tersebut termasuk

Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)
preprocessing karena mengubah struktur data mentah dari multi-output menjadi single-output agar sesuai dengan kebutuhan model regresi.

2. pada saat melakukan standard scaler yang berada pada pipeline :

Tabel 1. 2 Pipeline

```
('scaler', StandardScaler())
```

Standardisasi ini diterapkan pada variabel fitur (X) saat proses pipeline.fit(X_train, y_train) dijalankan. StandardScaler mengubah setiap fitur sehingga memiliki rata-rata 0 dan standar deviasi 1, dengan tujuan menyamakan skala antar fitur dan mencegah bias akibat perbedaan magnitude nilai. Langkah ini termasuk preprocessing karena mengubah representasi numerik data sebelum model melakukan proses pembelajaran. Untuk pipeline kami membuat menggunakan `sklearn.pipeline.Pipeline`:

Tabel 1. *3* sklearn pipeline

```
Pipeline([
    ('scaler', StandardScaler()),
    ('regressor', model)
])
```

Secara matematis, StandardScaler melakukan transformasi:

$$z = \frac{x - \mu}{\sigma}$$

$\mu =$ Mean
$\sigma =$ Standard Deviation

Gambar 1. 1 Transformasi StandardScaler

di mana μ adalah rata-rata fitur dan σadalah standar deviasi fitur. Transformasi ini memastikan setiap fitur berada pada skala yang sebanding sehingga proses optimisasi model menjadi lebih stabil secara numerik. Alasan penggunaan StandardScaler karena regresi linier sensitif terhadap skala fitur. Jika fitur memiliki skala berbeda:

- Koefisien menjadi tidak seimbang
- Model bias terhadap fitur bernilai besar
- Konvergensi bisa lebih lambat

Model seperti HuberRegressor menggunakan optimisasi iteratif berbasis minimisasi fungsi loss. Proses ini sensitif terhadap perbedaan skala fitur karena fitur dengan nilai sangat besar dapat mendominasi gradien dan memengaruhi proses konvergensi. Oleh karena itu, standardisasi fitur membantu meningkatkan stabilitas dan efisiensi proses pembelajaran model.

Scaling dilakukan hanya pada data train di setiap fold karena menggunakan Pipeline menghindari data leakage. Jika scaling dilakukan sebelum proses cross-validation, maka informasi statistik dari data uji (mean dan standar deviasi) dapat bocor ke data latih (data leakage). Hal ini dapat menyebabkan estimasi performa model menjadi terlalu optimistis dan tidak mencerminkan kemampuan generalisasi yang sebenarnya. Dengan mengintegrasikan preprocessing ke dalam Pipeline, validitas eksperimen tetap terjaga.

Preprocessing dalam kode ini bertujuan untuk:

1. Menyederhanakan struktur target menjadi satu variabel kontinu.

Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)

2. Menyamakan skala fitur agar model regresi bekerja secara optimal.
3. Menjaga validitas eksperimen melalui integrasi preprocessing dalam pipeline berbasis cross-validation.

Sehingga preprocessing dalam kode ini tidak hanya berfungsi sebagai tahap persiapan data, tetapi juga sebagai mekanisme untuk menjaga stabilitas numerik model, mencegah bias evaluasi, serta memastikan hasil eksperimen memiliki validitas metodologis yang kuat.

Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)

BAB II
METODOLOGI

2.1 Log Prompt

Pada tahap awal pengembangan (Prompt 1: Untuk memaksimalkan nilai evaluasi, saya ingin memperluas tabel log eksperimen. Tolong buatkan pipeline regresi menggunakan dataset SGEMM GPU dengan ketentuan berikut: Model: Bandingkan 5 model sekaligus, gunakan model yang paling sesuai dengan dataset sebagai baseline dan juga model yang lebih baik dan terbaru. Preprocessing: Gunakan StandardScaler yang dibungkus dalam Pipeline agar modular dan mencegah kebocoran data. Lakukan averaging pada 4 kolom target runtime menjadi satu target kontinu. Skenario Eksperimen: Buat dua fase pengujian:

- Fase Normal
- Fase Stress Test

Tampilkan hasil evaluasi (MAE, RMSE, MAPE, R2) dalam bentuk satu tabel ringkasan (Log Experiment) yang mencakup ke-10 skenario tersebut (5 model x 2 kondisi).) kode yang dihasilkan masih bersifat eksploratif dan mencakup lima model regresi linier, yaitu OLS, Ridge, Lasso, ElasticNet, dan Huber Regressor. Total eksperimen yang dijalankan berjumlah sepuluh, terdiri dari kondisi normal dan stress test untuk masing-masing model. Namun, pada tahap ini belum tersedia visualisasi hasil, tidak terdapat fitur ekspor log eksperimen ke file eksternal, serta belum menggunakan konfigurasi modular yang terstruktur.

Pada tahap kedua (Prompt 2 : dengan menggunakan codingan sebelumnya gunakan 2 model saja yaitu OLS dan Hubber Regressor, pastikan semuanya sesuai dengan ketentuan yang ada sebelumnya), fokus eksperimen disederhanakan menjadi perbandingan antara OLS sebagai baseline dan Huber Regressor sebagai model robust. Pipeline dibuat lebih bersih dan terstruktur, sementara mekanisme evaluasi menggunakan K-Fold Cross Validation tetap dipertahankan. Penyederhanaan ini bertujuan untuk memperjelas fokus analisis dan memperdalam perbandingan antar dua pendekatan utama. Pada versi final yang telah dimodifikasi, dilakukan beberapa perubahan secara manual yang dijelaskan pada keterbaruan.

2.2 Keterbaruan

Keterbaruan dalam penelitian ini dibandingkan implementasi regresi linier standar adalah sebagai berikut:

1) ***Clean Code***

Kode AI

Tabel 2. 1 Kode AI Keterbaruan 1

```
df = pd.read_csv('sgemm_product.csv')
```

Tidak ada manajemen konfigurasi, parameter langsung ditulis di fungsi

Modifikasi

Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)

Tabel 1. 4 Modifikasi Keterbaruan 1

```
CONFIG = {
        'DATA_PATH': 'sgemm_product.csv',
        'OUTLIER_PERCENTAGE': 0.05,
        'MULTIPLIER': 10,
        'LOG_FILENAME': 'log_experiment_final.csv',
        'PLOT_FILENAME': 'perbandingan_model.png'
}
```

Memberikan satu titik kendali (*Single Point of Control*). Jika ingin mengganti dataset, menaikkan persentase outlier, atau mengubah nama file output, kamu tidak perlu mencari-cari di ratusan baris kode.

2) **Visualisasi Data**

Kode AI

Tidak ada import matplotlib/seaborn dan tidak ada baris kode grafik

Modifikasi

Tabel 2. 2 Modifikasi Keterbaruan 1

```
print("grafik perbandingan")
  plt.figure(figsize=(10, 6))
  sns.barplot(data=df_log, x='Model', y='MAPE (%)', hue='Kondisi')
  plt.title('Perbandingan MAPE: Normal vs Stress Test', fontsize=14)
  plt.ylabel('MAPE (%) - Semakin Kecil Semakin Baik')
  plt.grid(axis='y', linestyle='--', alpha=0.7)
  plt.savefig(CONFIG['PLOT_FILENAME'])
  plt.show()
  print(f"[INFO] Grafik berhasil disimpan ke: {CONFIG['PLOT_FILENAME']}")
```

Digunakan untuk melihat model mana yang grafiknya paling stabil saat *Stress Test* tanpa perlu membandingkan angka desimal satu per satu.

3) ***Data Persistence***

Kode AI

Tidak ada kode AI, hanya print ke console.

Modifikasi

Tabel 2. 3 Modifikasi Keterbaruan 3

```
df_log.to_csv(CONFIG['LOG_FILENAME'], index=False)
print(f"\n[INFO] Tabel berhasil disimpan ke: {CONFIG['LOG_FILENAME']}")
```

Digunakan untuk menyimpan hasil ke CSV.

4) **Efisiensi Memori *(Optimization)***

Kode AI

Tabel 2. 4 Kode AI Keterbaruan 4

```
df = pd.read_csv('sgemm_product.csv')
```

Default float64 (boros RAM)

Modifikasi

Tabel 2. 5 Modifikasi Keterbaruan 4

```
df = pd.read_csv(CONFIG['DATA_PATH'], dtype=np.float32)
```

Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)

Penggunaan float32 bertujuan meningkatkan efisiensi memori dan memastikan eksperimen tetap ringan ketika dijalankan pada lingkungan komputasi terbatas.

5) **Fungsi Stress Test Manual (Injeksi Outlier)**

Kode AI

Tabel 2. 6 Kode AI Keterbaruan 5

```
for name, model in models.items():
    pipeline = build_pipeline(model)
    log_experiment.append(evaluate_experiment(f"{name} - Stress", pipeline, X, y))
```

AI biasanya hanya mengulang proses yang sama tanpa modifikasi data

Modifikasi

Tabel 2. 7 Modifikasi Keterbaruan 5

```
log_experiment.append(evaluate_experiment("Baseline OLS (Stress Test)",
pipeline_ols, X, y, inject_stress=True))
log_experiment.append(evaluate_experiment("Huber Regressor (Stress Test)",
pipeline_huber, X, y, inject_stress=True))
```

Digunakan untuk Pengujian Ketahanan (*Robustness Testing*) untuk melihat siapa yang paling kuat bertahan dalam kondisi terburuk.

Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)

BAB III
HASIL DAN PEMBAHASAN

3.1 Experiment Log

Sebelumnya kami mencoba untuk memastikan apa yang dikatakan dalam penelitian oleh (Ismail & Rasheed, 2021) yang menunjukan bahwa OLS tidak sebaik Huber Robust ketika dihadapkan pada kondisi data yang mengandung *outlier*. Untuk memverifikasi temuan tersebut secara empiris pada dataset SGEMM GPU Kernel Performance, pada tahap awal eksperimen kami menguji lima model regresi linier, yaitu OLS, Ridge, Lasso, ElasticNet, dan Huber Regressor. Tujuan pengujian awal ini adalah untuk memperoleh gambaran komparatif mengenai performa masing-masing model serta mengidentifikasi model yang paling sesuai dengan karakteristik dataset. Kelima model tersebut memiliki parameter sebagai berikut :

- OLS (Baseline): menggunakan parameter default dari sklearn tanpa regularisasi.
- Ridge (L2): menggunakan parameter default dengan penalti regularisasi L2.
- Lasso (L1): menggunakan alpha = 0.1 sebagai kekuatan regularisasi.
- ElasticNet: menggunakan alpha = 0.1 dengan kombinasi penalti L1 dan L2.
- Huber Regressor: menggunakan epsilon = 1.35 dan max_iter = 500 untuk mengontrol sensitivitas terhadap outlier dan jumlah iterasi optimisasi.

Tabel 3. 1 Experiment Log

No	Model	Kondisi	MAE (ms)	RMSE (ms)	MAPE (%)	R^2 Score	Catatan
1	OLS (Baseline)	Normal	176.4055	283.8648	214.2634	0.4073	Model dasar tanpa regularisasi
2	Ridge (L2)	Normal	176.4050	283.8648	214.2626	0.4073	Regularisasi L2
3	Lasso (L1)	Normal	176.3051	283.8648	214.0953	0.4073	Regularisasi L1
4	ElasticNet	Normal	172.3097	284.3067	207.4990	0.4055	Kombinasi L1 & L2
5	Huber (Robust)	Normal	143.3698	315.9265	122.8216	0.2659	Robust terhadap outlier
6	OLS (Baseline)	Stress Test	248.3017	318.7454	326.1999	0.2526	Sensitif terhadap outlier
7	Ridge (L2)	Stress Test	248.3008	318.7446	326.1989	0.2526	Lebih stabil dari OLS
8	Lasso (L1)	Stress Test	248.1985	318.6512	326.0477	0.2530	Penalti sparsity
9	ElasticNet	Stress Test	240.2086	312.4087	317.2543	0.2820	Stabilitas menengah

10	Huber (Robust)	Stress Test	144.7073	311.6241	131.8638	0.2858	Paling stabil terhadap outlier

Setelah memperoleh hasil ini kami memfokukan pada perbandingan antara OLS sebagai baseline dan Huber Regressor sebagai model robust, karena keduanya mewakili dua pendekatan yang secara teoritis berbeda dalam menangani error dan outlier.

3.2 Perbandingan Baseline Regresi Linier dengan Model Regresi Linier yang Terbaru

Baseline yang digunakan adalah Ordinary Least Squares (OLS). Model ini meminimalkan error kuadrat tanpa mempertimbangkan regularisasi maupun outlier. Ridge dan Lasso menambahkan penalti regularisasi untuk mengurangi overfitting dan meningkatkan generalisasi. ElasticNet menggabungkan kedua pendekatan tersebut. Huber Regressor menggunakan loss function yang lebih tahan terhadap outlier. Ketika error kecil, ia berperilaku seperti OLS, tetapi ketika error besar (outlier), ia menjadi lebih linear sehingga tidak terlalu terpengaruh nilai ekstrem. Pada kondisi normal, biasanya performa OLS dan model regularisasi tidak berbeda jauh jika data relatif bersih. Namun pada kondisi stress test, Huber cenderung mempertahankan nilai MAE, RMSE, dan MAPE yang lebih stabil dibanding OLS karena sifatnya yang robust terhadap outlier. Interpretasi umum hasil: Jika R2 turun drastis pada OLS saat stress test, tetapi Huber tetap stabil, maka dapat disimpulkan bahwa model robust lebih cocok untuk data dengan potensi outlier.

3.1 Hasil Evaluasi

Dalam regresi nilai MAE, RMSE, dan MAPE semakin kecil semakin baik sedangkan R^2 semakin besar semakin baik Pada kondisi normal (tanpa injeksi outlier), OLS, Ridge, dan Lasso menunjukkan performa hampir identik dengan R^2 sekitar 0.4073. Artinya model hanya mampu menjelaskan sekitar 40% variasi data, yang tergolong sedang. ElasticNet sedikit lebih rendah, sedangkan Huber memiliki R^2 lebih kecil (~0.2659). Hal ini menunjukkan bahwa ketika data relatif bersih, model linier klasik dan regularisasi bekerja hampir sama dan tidak terdapat overfitting yang signifikan.

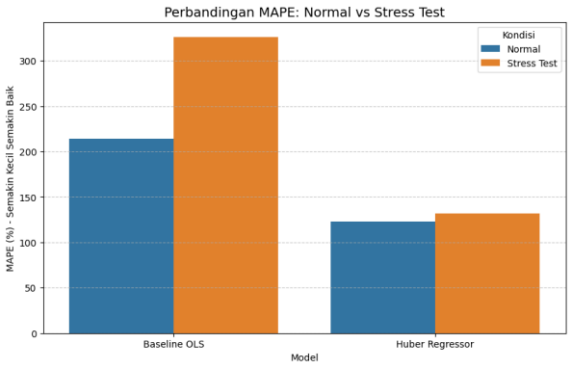
Namun pada kondisi stress test (injeksi 5% outlier), terjadi perubahan yang signifikan. Nilai R^2 OLS turun dari sekitar 0.40 menjadi sekitar 0.25. Penurunan ini terjadi karena OLS meminimalkan kuadrat error; ketika terdapat outlier, error besar akan dikuadratkan dan mendominasi proses pelatihan model. Ridge dan Lasso mengalami penurunan serupa karena regularisasi tidak dirancang untuk mengatasi outlier. Sebaliknya, Huber menunjukkan performa yang lebih stabil. Nilai R^2 tetap relatif lebih tinggi dibanding OLS pada kondisi stress test (sekitar 0.2858), dan nilai MAE serta MAPE meningkat jauh lebih kecil dibanding model lain. Secara kuantitatif, MAPE OLS melonjak hingga lebih dari 300%, sedangkan Huber bertahan di sekitar 131%. Hal ini membuktikan bahwa pendekatan robust lebih efektif dalam menghadapi data dengan nilai ekstrem.

Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)

BAB IV
STRESS TEST

4.1 Stress Test

Mekanisme *stress test* dilakukan untuk mengevaluasi ketahanan (*robustness*) berbagai model regresi terhadap anomali data yang ekstrem. Proses ini dimulai dengan menyalin dataset asli dan mengidentifikasi 5% sampel dari total data secara acak menggunakan fungsi sampling probabilitas. Pada sampel yang terpilih tersebut, nilai target waktu eksekusi GPU dimanipulasi dengan mengalikannya dengan faktor sepuluh guna menciptakan pencilan (*outliers*) buatan yang signifikan.



Gambar 4. 1 Perbandingan MAPE

Tujuan dari tahapan ini adalah untuk mensimulasikan kegagalan sensor atau kesalahan pencatatan data yang sering terjadi di dunia nyata, serta untuk membandingkan performa antara metode regresi standar seperti *Ordinary Least Squares* (OLS) dengan metode regresi tangguh seperti *Huber Regressor*. Melalui pengujian ini, efektivitas model dalam mempertahankan akurasi prediksi di bawah tekanan data yang terkontaminasi dapat diukur dan divalidasi secara objektif.

Laporan ini bertujuan untuk menyelesaikan tugas mata kuliah machine learning 1 yang ditulisakan oleh : Aliya Raffa Naura Ayu (2310817120014), Natalie Grace Katiandagho (2310817120003), Naila Hanifah (2310817220016)

DAFTAR PUSTAKA

DQLab, 2025. *Mengapa Data Preprocessing Penting dalam Machine Learnin.* [Online] Available at: https://dqlab.id/mengapa-data-preprocessing-penting-dalam-machine-learning [Accessed 27 February 2026].

GeeksforGeeks, 2026. *Data Processing in Machine learning.* [Online] Available at: https://www.geeksforgeeks.org/machine-learning/ml-understanding-data-processing/ [Accessed 27 February 2026].

Ismail, M. I. & Rasheed, H. A., 2021. Robust Regression Methods / a Comparison Study. *Turkish Journal of Computer and Mathematics Education.*

Formatted: English (United States)

Formatted: Normal, Line spacing: single